# The Comprehensive Guide to Gradient Descent and Feature Optimisation

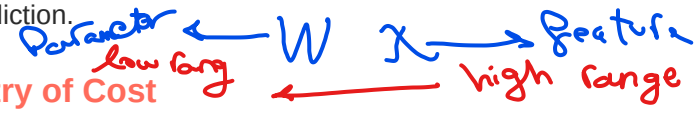
## Feature Scaling • Learning Rate Tuning • Convergence • Feature Engineering

*A practical reference for optimising machine learning models through disciplined data preparation and algorithmic tuning.*

## 1. Fundamentals of Feature Scaling

The scale of a feature directly influences the magnitude of its associated parameter (*w*). In a predictive model, the algorithm seeks parameter values that allow the weighted sum of features to match a target value (*y*).

If a feature has a large range (e.g., house size $x_1$ ranging from 300 to 2,000 sq. ft.), the model must learn a relatively small parameter $w_1$ (e.g., 0.1). This is because even a tiny increase in $w_1$ results in a massive surge in the predicted price, potentially overshooting the target y and inflating the cost J. Conversely, a feature with a small range (e.g., bedrooms $x_2$ ranging from 0 to 5) requires a much larger parameter $w_2$ (e.g., 50) to exert a meaningful influence on the prediction.




### Key Insight: The Geometry of Cost

Lopsided feature ranges create lopsided cost function contours. Because the horizontal and vertical axes represent weights of vastly different scales (e.g., one axis ranging from 0 to 1 and the other from 10 to 100), the contour plot forms tall, skinny ellipses. In this environment, gradient descent "bounces" back and forth across the narrow valley, taking an inefficient path to the minimum. By scaling features to comparable ranges, the contours become more circular, allowing gradient descent to follow a direct path to the global minimum.




## 2. Methods for Implementing Feature Scaling

To accelerate gradient descent, we transform features so they occupy similar ranges. The table below summarises the logic and typical resulting ranges for house-price features.

| Scaling Method | Calculation Logic | Typical Resulting Range |
|---|---|---|
| Max Scaling | Divide by maximum: x / x_max | $x_1$: 0.15 to 1 |
| Mean Normalisation | Centre around zero: (x – μ) / (max – min) | $x_1$: -0.18 to 0.82 (μ = 600) |
| Z-score Normalisation | Use standard deviation: (x – μ) / σ | $x_1$: -0.67 to 3.1 (μ = 600, σ = 450) |

**Note:** For the bedroom feature ($x_2$), Mean Normalisation yields a range of -0.46 to 0.54, while Z-score Normalisation results in -1.6 to 1.9 (μ = 2.3, σ = 1.4).

PYTHON

```python
# Z-score normalisation example
import numpy as np

mu = np.mean(X, axis=0)      # feature means
sigma = np.std(X, axis=0)    # feature std devs
X_norm = (X - mu) / sigma    # scaled features
```

## 3. Practical Rules of Thumb for Re-scaling

Feature scaling is not always strictly necessary, but it is rarely harmful. Use the following criteria to determine when to apply normalisation.

### Acceptable Ranges (Re-scaling Optional)

- -3 to +3
- -0.3 to +0.3
- 0 to 3
- -2 to +0.5

### Warning Signs (Re-scaling Recommended)

- **Large Ranges:** Features ranging from -100 to +100.
- **Extremely Small Values:** Ranges such as -0.001 to +0.001.
- **High-Magnitude Offsets:** Values that are large in magnitude but have small variances (e.g., body temperatures from 98.6 to 105°F). Even though the range (~6) is relatively small, the magnitude (~100) is high compared to other scaled features, which can slow down optimisation.

## 4. Monitoring Convergence with Learning Curves

A learning curve plots the Cost (J) against the Number of Iterations. This is a vital diagnostic tool to ensure the algorithm is descending the cost surface correctly.

1. **Consistent Decrease:** In a healthy implementation, the cost J should decrease after every single iteration. If it increases or oscillates, the learning rate is likely too high.
2. **Convergence Detection:** If the curve flattens out (e.g., after 400 iterations), the algorithm has converged. Note that the x-axis represents the iteration count, not a parameter like w or b.
3. **Variable Timelines:** Convergence can take 30 iterations or 100,000 depending on the application; the learning curve is the only reliable way to know when to stop.

### Automatic Convergence Test

You may implement an automatic test using a threshold variable epsilon ($\varepsilon$). If the cost J decreases by less than $\varepsilon$ (e.g., $10^{-3}$) in a single iteration, you can declare convergence. However, choosing the correct $\varepsilon$ is difficult; visual inspection of the learning curve remains the “gold standard” for senior specialists.

```python
prev_cost = compute_cost(X, y, w, b)
for i in range(max_iterations):
    w, b = gradient_descent_step(X, y, w, b, alpha)
    cost = compute_cost(X, y, w, b)
    if abs(prev_cost - cost) < epsilon:  # e.g. 1e-3
        print('Converged at iteration', i)
        break
    prev_cost = cost
```

## 5. Strategic Selection of the Learning Rate ($\alpha$)

The learning rate is the most critical hyperparameter. If $\alpha$ is too small, gradient descent is painfully slow. If it is too large, the algorithm may overshoot the minimum, causing the cost to increase or oscillate.

### Step-by-Step Debugging Protocol

1. **The "Plus Sign" Bug Check:** Set $\alpha$ to an extremely small number. If the cost J does not decrease consistently, check your update rule for a sign error (e.g., using $w = w + \alpha \cdot \partial J/\partial w$ instead of the correct minus sign).
2. **The Roughly 3x Increment Strategy:** Systematically test values by increasing them threefold to find the boundary of stability.
3. **Optimal Selection:** Select the largest stable learning rate that yields a consistently decreasing cost.

### Recommended Test Sequence

- 0.001
- 0.003
- 0.01
- 0.03
- 0.1 (and so on)

```
TERMINAL SESSION
$ python train.py --alpha 0.001   # too slow
$ python train.py --alpha 0.01    # better
$ python train.py --alpha 0.1     # cost diverges!
$ python train.py --alpha 0.03    # stable & fast
```

## 6. Advanced Performance: Feature Engineering

Feature engineering allows you to use intuition to transform original features, enabling the model to fit non-linear curves rather than just straight lines.

### Case Study: Property Area Engineering

Consider a model using frontage ($x_1$) and depth ($x_2$) of a plot: $f(x) = w_1x_1 + w_2x_2 + b$. While the model could use these separately, your intuition might suggest that total area is a better predictor of price. By creating a new engineered feature $x_3 = x_1 \times x_2$ (area), the model becomes: $f(x) = w_1x_1 + w_2x_2 + w_3x_3 + b$. This allows the algorithm to adjust $w_1$, $w_2$, and $w_3$ to determine if frontage, depth, or the total area is the most predictive factor, significantly improving the fit for non-linear data relationships.

PYTHON

```python
# Engineering an 'area' feature
frontage = X[:, 0]
depth = X[:, 1]
area = frontage * depth          # new feature x3

X_engineered = np.column_stack([frontage, depth, area])
```

## 7. Final Summary Checklist

- **Feature Scaling:** Ensure features occupy comparable ranges (ideally -1 to +1) to avoid “bouncing” on elliptical contours.
- **Initial Debugging:** Run with a very small $\alpha$ to ensure the cost decreases, ruling out update rule bugs.
- **Learning Rate Selection:** Use the 3x increment strategy to find the largest stable rate before the cost increases or oscillates.
- **Convergence Monitoring:** Use learning curves (J vs Iterations) to verify the algorithm has flattened at the global minimum.
- **Feature Engineering:** Evaluate if combining features (like calculating 'area' from 'width' and 'depth') could better capture non-linearities in your data.

**Takeaway:** Gradient descent performs best when features are scaled to comparable ranges, the learning rate is tuned methodically via the 3x strategy, convergence is verified visually with a learning curve, and thoughtful feature engineering is used to capture non-linear relationships.